



# Alexandria National University

## Social Networks Report

*Prepared by:*

- Amina Ahmed Ferra – 2205225

```

!pip install torch_geometric

import torch
from torch_geometric.data import Data
from torch_geometric.nn import SAGEConv
import torch.nn.functional as F

# 0,1,1,1]

```

PyTorch and PyTorch Geometric modules are loaded to handle graph data and build GraphSAGE layers.

```

#--- Define a small graph with 6 nodes ---
# Node features (2 features per node).
# Here benign users have [1, 0] and malicious have [0, 1] for illustration.
x = torch.tensor(
    [
        [1.0, 0.0], # Node 0 (benign)
        [1.0, 0.0], # Node 1 (benign)
        [1.0, 0.0], # Node 2 (benign)
        [0.0, 1.0], # Node 3 (malicious)
        [0.0, 1.0], # Node 4 (malicious)
        [0.0, 1.0] # Node 5 (malicious)
    ],
    dtype=torch.float,
)

```

6 nodes are defined, each with 2 features.  
 Feature [1, 0] represents benign, [0, 1] represents malicious.

```

# Edge List (undirected). Connect benign users (0-1-2 fully connected)
# and malicious users (3-4-5 fully connected), plus one cross-edge 2-3.
edge_index = (
    torch.tensor(
        [
            [0, 1],
            [1, 0],
            [1, 2],
            [2, 1],
            [0, 2],
            [2, 0],
            [3, 4],
            [4, 3],
            [4, 5],
            [5, 4],
            [3, 5],
            [5, 3],
            [2, 3],
            [3, 2], # one connection between a benign (2) and malicious (3)
        ],
        dtype=torch.long,
    )
    .t()
    .contiguous()
)

```

Edges represent connections in the graph.

(0, 1, 2) nodes are fully connected among themselves. (3, 4, 5) nodes are also fully connected.

The graph is undirected, so each edge is listed twice.

```

y = torch.tensor([0, 0, 0, 1, 1, 1], dtype=torch.long)

data = Data(x=x, edge_index=edge_index, y=y)

# --- Define a two-layer GraphSAGE model ---
# his defines a 2-layer GraphSAGE neural network.
# in_channels=2 means each node has 2 features.
# hidden_channels=4 creates a 4-dimensional hidden embedding.
# out_channels=2 means the model outputs scores for 2 classes (benign and malicious).

class GraphSAGENet(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels):
        super(GraphSAGENet, self).__init__()
        self.conv1 = SAGEConv(in_channels, hidden_channels)
        self.conv2 = SAGEConv(hidden_channels, out_channels)

    def forward(self, x, edge_index):
        # First layer: sample neighbors and aggregate
        x = self.conv1(x, edge_index)
        x = F.relu(x) # non-linear activation
        # Second layer: produce final embeddings/class scores
        x = self.conv2(x, edge_index)
        return F.log_softmax(x, dim=1) # log-probabilities for classes

```

Labels indicate class per node: 0 for benign, 1 for malicious.

Features, edges, and labels are combined into a PyTorch Geometric Data object.

A two-layer GraphSAGE neural network is defined.

The first layer transforms input features into a 4-dimensional embedding.

The second layer outputs scores for two classes.

ReLU activation is applied between layers, and log-softmax produces class probabilities.

```

# Instantiate model: input dim=2, hidden=4, output dim=2 (benign vs malicious)
model = GraphSAGENet(in_channels=2, hidden_channels=4, out_channels=2)

# Simple training loop
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
model.train()
for epoch in range(50):
    optimizer.zero_grad()
    out = model(data.x, data.edge_index)
    loss = F.nll_loss(out, data.y) # negative log-likelihood
    loss.backward()
    optimizer.step()

```

Model training runs for 50 epochs.

Each epoch includes resetting gradients, forward pass, loss calculation using negative log-likelihood, backpropagation, and optimizer step to update weights.

```
# After training, we can check predictions
model.eval()
pred = model(data.x, data.edge_index).argmax(dim=1)
print("Predicted labels:", pred.tolist()) # e.g. [0,0,
7]

Predicted labels: [0, 0, 0, 1, 1, 1]
```

Model switches to evaluation mode.

Predictions are made by taking the class with the highest score for each node.

Predicted labels are printed as a list.